
第 8 章

Sklearn 及其基本使用

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

8. scikit-learn 及其基本使用（新版讲义）

本页是第 8 章的**章首页（索引）**。正文位于 01-数据集的预处理.md ~ 05-常见误区与技巧.md，与原版资料完全分离（原版见 ../../原始资料/chap8/）。

本章概览

scikit-learn（简称 sklearn）是 Python 最常用的机器学习库：它把“数据预处理 → 模型训练 → 模型评估 → 模型调参 → 可视化”这一整套流程统一成一套简洁、一致的接口（fit / predict / score / transform）。本章以经典数据集（Iris 鸢尾花、Wine 红酒、Diabetes 糖尿病、乳腺癌等）为例，带你从**特征编码、缩放、切分、交叉验证、特征选择**讲起，再分别讲解**有监督学习**（分类、回归）与**无监督学习**（KMeans/DBSCAN 聚类、PCA/LDA 降维），最后用一个 Wine 端到端案例把全部知识串起来。

sklearn 聚焦“经典机器学习”，接口统一、算法丰富、与 NumPy/Pandas/Matplotlib 无缝配合，是《Python 科学计算》课程最后一块重要拼图。

学习目标

学完本章，你应该能够：

1. 说清**有监督学习**（分类、回归）与**无监督学习**（聚类、降维）的区别，并能在 sklearn 中选对模型。
2. 用 OneHotEncoder / LabelEncoder 做特征编码，用 StandardScaler / MinMaxScaler 做特征缩放，理解“为什么距离类算法要缩放”。
3. 用 train_test_split 切分数据，并用 cross_val_score / LeaveOneOut 评估模型泛化能力，避免数据泄漏。
4. 用 VarianceThreshold、SelectKBest、RFE、SelectFromModel、Lasso 做特征筛选。
5. 完成一个分类任务（Iris/乳腺癌）与一个回归任务（Diabetes），会用 accuracy_score、classification_report、roc_auc_score、mean_squared_error、r2_score 等指标评估。
6. 完成 KMeans 与 DBSCAN 聚类、PCA 与 LDA 降维，会用 silhouette_score、adjusted_rand_score、explained_variance_ratio_ 评估结果。
7. 用 Pipeline + ColumnTransformer 把预处理与模型串成一条流水线，避免训练集/测试集信息泄漏。
8. 完成一个 Wine 端到端案例：预处理 → 分类 → 聚类 → 降维 → 可视化，并读懂其中的特征重要性。

先修要求与运行环境

- 熟悉 Python 基础语法；建议先学完本课程第 1–6 章（NumPy、SymPy、SciPy、Pandas、Matplotlib、NetworkX）。
- 了解基本的矩阵/向量代数与统计概念（均值、方差、相关系数等）即可，不要求先修研究生水平的机器学习课程。
- 安装与升级（建议 Python 3.10+）：

```
pip install --upgrade pip
pip install numpy pandas matplotlib scipy scikit-learn
# 第 7 章统计建模（可选）
pip install statsmodels -i https://pypi.tuna.tsinghua.edu.cn/simple
```

- 推荐在 JupyterLab / VS Code 中打开 .ipynb 练习；sklearn ≥ 1.3 即可运行本章代码（本仓库在 sklearn 1.6.1 下验证）。

案例索引

案例卡	所在节	技术点	难度
1. 特征编码与缩放——为什么先 fit 再 transform	01 数据集的预处理	OneHotEncoder、StandardScaler、fit/transform、防泄漏	★★☆
2. 分类流水线——Iris 用 KNN+ 标准化 +Pipeline 防数据泄漏	02 有监督学习的案例	Pipeline、StandardScaler、KNN、cross_val_score	★★☆
3. KMeans 与 PCA——用肘部法则评估聚类数并做 2D 可视化	03 无监督学习的案例	KMeans、inertia、PCA、silhouette/ARI	★★☆
4. Wine 数据端到端——预处理 → 分类 → 聚类 → 降维 → 可视化	04 综合案例	Pipeline、特征重要性、KMeans、PCA	★★★

本章目录

小节	文件	内容
01 数据集的预处理	01-数据集的预处理.md	特征编码、特征缩放、数据集切分、交叉验证、特征选择与降维概览（案例卡 1）
02 有监督学习的案例	02-有监督学习的案例.md	Iris 分类、乳腺癌二分类、Diabetes 回归、决策树可视化、评估指标（案例卡 2）
03 无监督学习的案例	03-无监督学习的案例.md	KMeans / DBSCAN 聚类、PCA / LDA 降维、聚类与降维评估（案例卡 3）
04 综合案例	04-综合案例.md	Wine 数据端到端：预处理 → 分类 → 聚类 → 降维 → 可视化（配图，案例卡 4）
05 常见误区与技巧	05-常见误区与技巧.md	易错点表格、性能/调试/自测清单

数学预备与附录

本章的机器学习背景已集中到附录《数学与算法补充》：

- **E 机器学习基础**：E.1 学习范式、E.2 损失与优化、E.3 评估与过拟合、E.4 常用模型直觉、E.5 sklearn 流程；
 - **A 线性代数与矩阵**：A.3 SVD/PCA（降维）；
 - [打开附录 E](#) | [打开附录 A](#) | [附录索引](#)
- 课堂先讲 E.1~E.3（约 15 分钟），再进各模型；PCA 课回看 A.3。

练习与上机入口

- [本章练习\(exercises/\)](#)：10 道自测 quiz(自动评分)+ 一章作业 assignment.md + 参考答案 answers.ipynb。
- [本章上机 \(lab/\)](#)：循序渐进的上机 notebook（Part 1–4），含综合任务与提交清单。
- [本章参考与延伸阅读 \(references.md\)](#)：官方文档、精品教程、习题实战、中文补充。
- [教学说明 \(teaching.md\)](#)：课时安排、重点难点、考核建议（教师用）。

建议课时

环节	学时	对应内容
讲课	4-5 学时	01-03 正文 + 04 综合案例讲解
上机	3-4 学时	lab/ 逐题完成; 课后完成 exercises/ 作业

使用说明

- **学生**: 先读 01-03 正文并运行代码 → 完成 lab 上机 → 做 exercises 作业 → 自测 quiz 检验。生词/公式不懂时, 到 references.md 找官方对应章节。
- **教师**: 按 teaching.md 的课时表讲; lab 可作为上机课内容; exercises 中的 quiz 带自动评分, 可直接回收。

01 数据集的预处理

本节是从“原始数据”到“可训练数据”的第一步：特征编码、特征缩放、数据集切分、交叉验证与特征选择。所有代码在 sklearn 1.6.1、Python 3.13 下运行验证。

本节目标

1. 会用 OneHotEncoder / LabelEncoder 处理分类特征，并说清两者适用场景。
2. 会用 StandardScaler / MinMaxScaler 缩放连续特征。
3. 会用 train_test_split 切分数据，并用 cross_val_score / LeaveOneOut 做交叉验证。
4. 会用 VarianceThreshold、SelectKBest、RFE、Lasso、SelectFromModel 做特征选择。
5. 理解预处理必须放在 Pipeline 中，避免数据泄漏。

先修

- NumPy 数组操作、Pandas 基本用法（第 1、4 章）。
- 了解“均值/方差/相关系数”与“特征量纲”概念。

官方文档 / 参考入口

- sklearn 预处理：[链接](#)
- sklearn 模型选择：[链接](#)
- sklearn 特征选择：[链接](#)
- sklearn 数据分割 API：[链接](#)

正文

1. 特征编码：把“文字”变成“数字”

大多数 sklearn 模型只能吃数值矩阵。对待分类特征，有两种常见编码。

1.1 独热编码 (OneHotEncoder) 把每个类别展开成一位为 1、其余为 0 的向量。默认输出稀疏矩阵，用 `sparse_output=False` 得到稠密数组。

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import OneHotEncoder

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

data = np.array([[ 'male' ], [ 'female' ], [ 'male' ], [ 'female' ]])
encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(data)
print("categories_:", encoder.categories_)
print("shape:", encoded.shape)
print("特征名:", encoder.get_feature_names_out())
print(encoded)
```

```
categories_: [array(['female', 'male'], dtype='<U6')]
shape: (4, 2)
特征名: ['x0_female' 'x0_male']
[[0. 1.]
 [1. 0.]
 [0. 1.]
 [1. 0.]]
```

注意: male 被编码为 [0,1] (因为 categories_ 按字母序把 female 排在 0 列、male 排 1 列)。独热编码没有“谁大谁小”的含义, 适合无序类别。

1.2 标签编码 (LabelEncoder) 把每个类别映射成 0、1、2 等整数。它暗示类别之间有顺序, 对无序类别 (如颜色、性别) 通常不合适。

```
from sklearn.preprocessing import LabelEncoder
```

```
labels = ['male', 'female', 'male', 'female']
le = LabelEncoder().fit(labels)
print("classes_:", le.classes_)
print("transform:", le.transform(labels))

classes_: ['female' 'male']
transform: [1 0 1 0]
```

若类别本身有序 (如 小/中/大), 可改用 OrdinalEncoder 并显式指定 categories; 若只是把字符串标签喂给分类器, 常用 LabelEncoder 只看目标 y, 而特征 x 的分类列建议用 OneHotEncoder 或 OrdinalEncoder。

2. 特征缩放: 把量纲拉平

当各列量纲差别很大 (比如一列在 0~1, 另一列在 10^4 量级) 时, 距离类算法 (KNN、SVM、KMeans、PCA) 会被大方差特征主导, 因此需要缩放。

2.1 标准化 (StandardScaler) $x' = (x - \text{mean}) / \text{std}$, 使每列均值 0、标准差 1。

```
import numpy as np
from sklearn.preprocessing import StandardScaler

X = np.array([[1, 2], [3, 4], [5, 6]])
ss = StandardScaler().fit(X)
print("mean_:", ss.mean_, "scale_:", np.round(ss.scale_, 6))
print(ss.transform(X))

mean_: [3. 4.] scale_: [1.633 1.633]
[[-1.2247 -1.2247]
 [ 0.      0.     ]
 [ 1.2247  1.2247]]
```

2.2 归一化 (MinMaxScaler) 把每列压缩到 [0, 1] (或其他 feature_range)。

```
from sklearn.preprocessing import MinMaxScaler

mm = MinMaxScaler().fit(X)
print("data_min_:", mm.data_min_, "data_max_:", mm.data_max_)
print(mm.transform(X))
```

```
data_min_: [1. 2.] data_max_: [5. 6.]  
[[0.  0. ]  
 [0.5 0.5]  
 [1.  1. ]]
```

标准化对异常值更稳健 (用均值/标准差), 归一化把数据严格限制到区间。若有人告诉你 “某个算法必须归一化”, 指的是这两者之一; 具体选哪个要看数据分布与模型 (树模型一般不需要缩放)。

3. 数据集切分与交叉验证

3.1 train_test_split

```
from sklearn.model_selection import train_test_split  
from sklearn.datasets import load_iris  
  
iris = load_iris(); X, y = iris.data, iris.target  
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)  
print("train shape:", Xtr.shape, "test shape:", Xte.shape)  
print(" 测试集各类数量:", np.unique(yte, return_counts=True))  
  
train shape: (105, 4) test shape: (45, 4)  
测试集各类数量: (array([0, 1, 2]), array([15, 15, 15]))
```

stratify=y 让训练/测试集各类比例一致 (对分类很重要); random_state=42 保证可复现。

3.2 K 折交叉验证 (cross_val_score) 把数据切成 K 份, 轮流用 K-1 份训练、1 份测试, 最后平均各折分数。

```
from sklearn.linear_model import LogisticRegression  
from sklearn.model_selection import cross_val_score  
  
model = LogisticRegression(max_iter=1000)  
scores = cross_val_score(model, Xtr, ytr, cv=5)  
print(" 各折分数:", np.round(scores, 4))  
print(" 平均:", round(scores.mean(), 4))  
  
各折分数: [0.9524 0.9524 1.      0.9524 1.      ]  
平均: 0.9714
```

cross_val_score 内部会把每个模型 fit 在训练折、score 在验证折, 因此它比单次 train/test 更稳健、也更少受随机划分影响。

3.3 留一交叉验证 (LeaveOneOut) 每次只留 1 个样本测试。适合小数据集, 但计算量大。

```
from sklearn.model_selection import LeaveOneOut  
  
loo = LeaveOneOut()  
scores = []  
for tr, te in loo.split(Xtr[:30], ytr[:30]):  
    m = LogisticRegression(max_iter=1000).fit(Xtr[tr], ytr[tr])  
    scores.append(m.score(Xtr[te], ytr[te]))  
print(" 留一法样本数:", len(scores), " 平均:", round(np.mean(scores), 4))  
  
留一法样本数: 30 平均: 0.9333
```

4. 特征选择: 保留更少的列

特征选择从原始列中挑出 “最重要” 的子集, 能降低过拟合、加速训练。

4.1 过滤法: VarianceThreshold (方差阈值)

```
from sklearn.feature_selection import VarianceThreshold
```

```
sel = VarianceThreshold(threshold=0.8)
Xv = sel.fit_transform(X)
print(" 方差:", np.round(sel.variances_, 4))
print(" 变换后 shape:", Xv.shape)
```

```
方差: [0.6811 0.1887 3.0955 0.5771]
变换后 shape: (150, 1)
```

只保留方差 ≥ 0.8 的特征 (这里只有第 3 列)。方差小的列信息量低; 但注意它只按方差筛, 完全不看与 y 的关系。

4.2 过滤法: SelectKBest + 互信息

```
from functools import partial
from sklearn.feature_selection import SelectKBest, mutual_info_classif
```

```
selb = SelectKBest(partial(mutual_info_classif, random_state=42), k=2)
Xb = selb.fit_transform(X, y)
print(" 选中的特征索引:", selb.get_support(indices=True))
print(" 各特征互信息:", np.round(selb.scores_, 4))
print(" 变换后 shape:", Xb.shape)
```

```
选中的特征索引: [2 3]
各特征互信息: [0.5114 0.2994 0.9926 0.9856]
变换后 shape: (150, 2)
```

4.3 包装法: RFE (递归特征消除)

```
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE
```

```
est = LogisticRegression(max_iter=1000)
r = RFE(est, n_features_to_select=2)
Xr = r.fit_transform(X, y)
print("support_:", r.support_)
print("ranking_:", r.ranking_)
print(" 变换后 shape:", Xr.shape)
```

```
support_: [False False  True  True]
ranking_: [3 2 1 1]
变换后 shape: (150, 2)
```

4.4 嵌入法: Lasso / SelectFromModel Lasso 的 L1 正则会把不重要的系数压到 0; 用 SelectFromModel 包一个树模型, 可按重要性阈值自动选列。

```
from sklearn.linear_model import Lasso
```

```
ls = Lasso(alpha=0.1).fit(X, y)
print("Lasso 系数:", np.round(ls.coef_, 6))
print(" 非零掩码:", ls.coef_ != 0)
```

```
Lasso 系数: [ 0.      -0.      0.4081  0.      ]
非零掩码: [False False  True False]
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel

rf = RandomForestClassifier(n_estimators=10, random_state=42).fit(X, y)
selm = SelectFromModel(rf, prefit=True, threshold='mean')
Xm = selm.transform(X)
print(" 重要性:", np.round(rf.feature_importances_, 4), " 均值:",
      round(rf.feature_importances_.mean(), 4))
print("support_:", selm.get_support())
print(" 变换后 shape:", Xm.shape)

重要性: [0.1293 0.0158 0.4447 0.4102]  均值: 0.25
support_: [False False  True  True]
变换后 shape: (150, 2)
```

5. 特征降维一览

特征降维是把高维空间压到低维（如 2 维便于可视化）。sklearn 常用：

- **PCA**（无监督）：找方差最大的方向，示例见 03 节。
- **LDA**（有监督）：融合类别信息，最大化类间/类内方差之比，示例见 03 节。

案例卡 1：特征编码与缩放——为什么先 fit 再 transform

目标

用 OneHotEncoder 把无序文字类别变成 0/1 向量，用 StandardScaler 把不同量纲的数值列拉平；并说清 fit 学统计量、transform 应用统计量，以及为什么训练/测试要分开 fit/transform。

代码

```
import numpy as np
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris

np.set_printoptions(precision=4, suppress=True)

# (1) OneHotEncoder 编码无序分类特征
cats = np.array([[ 'male', 'low'], [ 'female', 'mid'], [ 'male', 'high'], [ 'female', 'mid']])
enc = OneHotEncoder(sparse_output=False).fit(cats)
print('类别组合:', enc.categories_)
print('编码后形状:', enc.transform(cats).shape)
print(enc.transform(cats))

# (2) StandardScaler: fit 学统计量, transform 应用
X = np.array([[1.0, 10.0], [2.0, 20.0], [3.0, 30.0], [4.0, 40.0]])
ss = StandardScaler().fit(X)
print('mean_:', ss.mean_, 'scale_:', np.round(ss.scale_, 4))
print('标准化结果:')
print(np.round(ss.transform(X), 4))

# (3) 为什么先 fit 再 transform: 用训练集统计量处理测试集
iris = load_iris(); X, y = iris.data, iris.target
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
sc = StandardScaler().fit(Xtr)
Xte_s = sc.transform(Xte)
print('训练集 mean_: ', np.round(sc.mean_, 4))

# 反例: 对全量 fit_transform(泄漏) 后, 测试集被全量统计量影响
sc_all = StandardScaler().fit(X)
Xte_leak = sc_all.transform(Xte)
print('正确做法测试集均值:', np.round(Xte_s.mean(axis=0), 4))
print('泄漏做法测试集均值:', np.round(Xte_leak.mean(axis=0), 4))
```

运行结果 (已运行核验)

```
类别组合: [array(['female', 'male'], dtype='<U6'), array(['high', 'low', 'mid'],
dtype='<U6')]
编码后形状: (4, 5)
[[0. 1. 0. 1. 0.]
 [1. 0. 0. 0. 1.]
 [0. 1. 1. 0. 0.]
 [1. 0. 0. 0. 1.]]
mean_: [ 2.5 25. ] scale_: [ 1.118 11.1803]
标准化结果:
[[-1.3416 -1.3416]
 [-0.4472 -0.4472]
 [ 0.4472  0.4472]
 [ 1.3416  1.3416]]
训练集 mean_: [5.8733 3.0552 3.7848 1.2057]
正确做法测试集均值: [-0.1164  0.0153 -0.0502 -0.0275]
泄漏做法测试集均值: [-0.0848  0.0113 -0.0355 -0.0196]
```

讲解

一句话解释: fit 是 “量尺寸”, transform 是 “按尺寸改衣服”; 尺寸只能从训练集量, 不能把测试集也一起量, 否则就 “作弊” 了。

1. OneHotEncoder 把两列文字展开成 5 列 (性别 2 列 + 等级 3 列), 每行只有 1 个位置为 1, 其余为 0; 它**不关心谁大谁小**, 适合无序类别。
2. StandardScaler().fit(X) 只做一件事: 算出每列的均值 mean_ 和标准差 scale_, 这里 mean_ = [2.5, 25.]、scale_ = [1.118, 11.1803]。
3. ss.transform(X) 按公式 $(x - \text{mean}) / \text{scale}$ 逐元素变换, 得到每列均值约 0、标准差约 1。
4. 第 (3) 步用 Iris 演示: 先在训练集上 fit, 得到训练集均值 [5.8733, 3.0552, 3.7848, 1.2057]; 再对测试集 transform, 测试集均值不为 0, 这是**正常的** (测试集不应使用自己的统计量)。
5. 最后两行对比 “泄漏做法”: 如果对全量数据 fit 后再 transform 测试集, 测试集均值会偏向全量统计量 (-0.0848 与 -0.1164 不同)。训练/测试切分前就做缩放, 等于让模型提前 “见过” 测试集的分布, 真实评估会被高估。

主要用法 / API

需求	写法	说明
无序类别编码	<code>OneHotEncoder(sparse_output=False)</code>	输出稠密矩阵，适合打印/调试
连续特征缩放	<code>StandardScaler()</code> 或 <code>MinMaxScaler()</code>	拉平量纲
学统计量	<code>scaler.fit(X_train)</code>	只使用训练集
应用统计量	<code>scaler.transform(X_test)</code>	用训练集统计量
一步完成	<code>scaler.fit_transform(X)</code>	只适合训练集或全量一次性转换
组合更安全	<code>ColumnTransformer + Pipeline</code>	保证交叉验证每折都重新 fit

常见错误

- 对测试集调用 `fit_transform` → 信息泄漏；应先把 `fit` 用在训练集。
- 把 `LabelEncoder` 当成特征编码 → 无序类别被强加虚拟顺序。
- 传给 `OneHotEncoder` 用旧参数 `sparse=False` → 新版本应为 `sparse_output=False`。
- 缩放后只打印变换结果、不打印 `mean_/scale_`，很难发现是否用错了统计量。

拓展 / 跟练

1. 把 `StandardScaler` 换成 `MinMaxScaler`，比较变换后的最小/最大值。
2. 用 `ColumnTransformer` 同时处理 `cats` 的分类列与 `X` 的数值列，打印变换后形状。
3. 在 02 节用 `Pipeline` 包住“缩放 + 模型”跑 5 折，观察与手写 `fit/transform` 的差别。
4. 给 `cats` 增加一个训练时没见过的类别 `male/unknown`，看 `transform` 会如何报错。

常见误区

1. 用 `fit_transform` 处理测试集 → 应先在训练集 `fit`，再对训练/测试都用 `transform`；最稳的做法是放进 `Pipeline`。
2. 用 `LabelEncoder` 编码无序分类特征 → 会引入虚假顺序；无序类别建议 `OneHotEncoder`。
3. 分类数据不编码直接喂 `LogisticRegression` → 会报错或产生错误结果。
4. 对树模型强行缩放，既不必要也浪费算力（通常无收益）。
5. `cross_val_score` 里把整个数据（含测试集）都用来调参 → 信息泄漏；网格搜索要放在独立验证集或嵌套交叉验证中。

思考题

1. 为什么独热编码会增加特征维度？类别很多时会产生什么问题（维度灾难/稀疏）？
2. 什么情况下 `MinMaxScaler` 比 `StandardScaler` 更好？什么情况相反？
3. 用 `LeaveOneOut` 时，为什么样本多时开销巨大？
4. `VarianceThreshold` 与 `SelectKBest` 的思路有何不同？

动手练习

1. 对 `['red', 'green', 'blue']` 做 `OneHot` 与 `Label` 编码，比较输出。
2. 对 `make_blobs`（中心 3 个）做 `StandardScaler` 后 `KMeans`，比较缩放前后 `silhouette_score`。
3. 在 `Iris` 上用 `SelectKBest` 选 3 个特征做 `cross_val_score`，与用全部特征比较。

延伸阅读

- sklearn 预处理用户指南: [链接](#)
- sklearn 特征选择用户指南: [链接](#)
- sklearn Pipeline 用户指南: [链接](#)

02 有监督学习的案例

本节在两个经典任务上走一遍“加载数据 → 切分 → 训练 → 评估”：**分类**（Iris 与乳腺癌）与**回归**（Diabetes）。所有代码在 sklearn 1.6.1 下验证。

本节目标

1. 用 `train_test_split` + 多个分类器完成 Iris 分类，并读懂混淆矩阵与分类报告。
2. 用 `Pipeline` + `StandardScaler` 训练逻辑回归，在乳腺癌上给出 `roc_auc_score`。
3. 用 `plot_tree` 可视化一棵决策树。
4. 在 Diabetes 数据集上做线性回归，比较 `LinearRegression`、`Ridge`、`Lasso`。
5. 会用 `mean_squared_error`、`mean_absolute_error`、`r2_score` 评估回归。

先修

- 01 节的数据预处理（缩放、切分、交叉验证）。
- 基本统计概念（均值、方差、相关系数）。

官方文档 / 参考入口

- sklearn 分类器总览：[链接](#)
- sklearn 模型评估：[链接](#)
- sklearn 决策树：[链接](#)
- sklearn 线性模型：[链接](#)

正文

1. 分类：Iris 鸢尾花

鸢尾花数据集有 150 个样本、4 个特征（花萼长宽、花瓣长宽），标签为 3 类（Setosa / Versicolor / Virginica），每类 50 条。

1.1 加载数据并切分

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

```
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)
```

```
iris = load_iris()
X, y = iris.data, iris.target
print(" 特征名:", iris.feature_names)
print(" 标签名:", iris.target_names)
print(" 数据形状:", X.shape)
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
print("train/test:", Xtr.shape, Xte.shape)
```

Iris 数据集特征两两分布

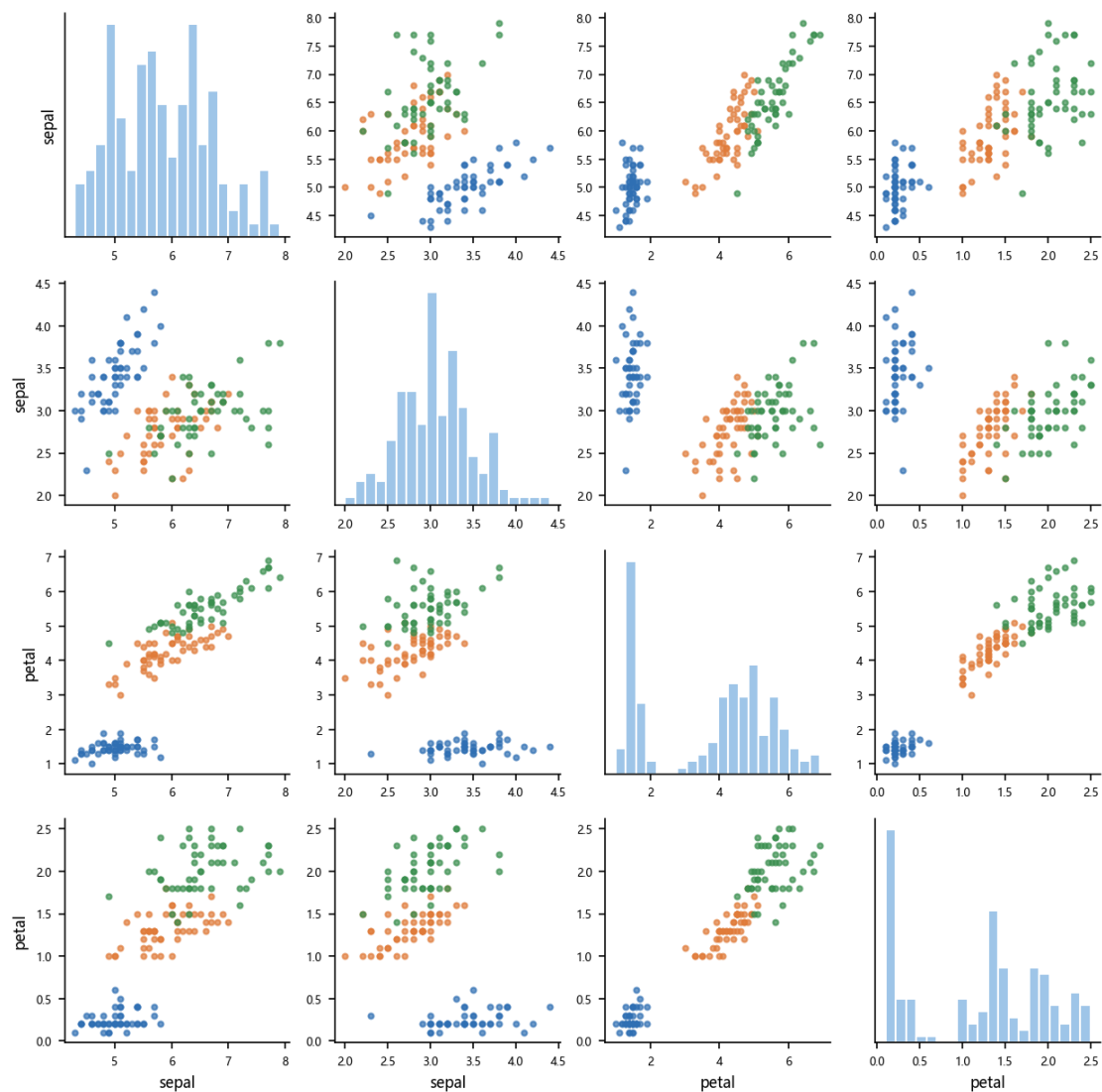


Figure 1: Iris 特征两两分布

特征名: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
 标签名: ['setosa' 'versicolor' 'virginica']
 数据形状: (150, 4)
 train/test: (105, 4) (45, 4)

1.2 训练多个分类器并比较

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

models = {
    "KNN(k=5)": KNeighborsClassifier(n_neighbors=5),
    "LogisticRegression": LogisticRegression(max_iter=1000),
    "DecisionTree": DecisionTreeClassifier(random_state=42),
    "RandomForest(100)": RandomForestClassifier(n_estimators=100, random_state=42),
}
for name, m in models.items():
    m.fit(Xtr, ytr)
    print(f"{name:22s} acc = {accuracy_score(yte, m.predict(Xte)):.4f}")

KNN(k=5)          acc = 0.9778
LogisticRegression acc = 0.9333
DecisionTree       acc = 0.9333
RandomForest(100)  acc = 0.8889
```

不同模型在同一数据上表现不同，没有对任何数据集都最优的“万能模型”。随机森林在这份固定切分上反而略低，说明小样本下模型选择要看交叉验证与多次随机切分，而非一次 test 结果。

1.3 混淆矩阵与分类报告

```
from sklearn.metrics import confusion_matrix, classification_report

rf = models["RandomForest(100)"]
print(confusion_matrix(yte, rf.predict(Xte)))
print(classification_report(yte, rf.predict(Xte), target_names=iris.target_names))
```

```
[[15  0  0]
 [ 0 14  1]
 [ 0  4 11]]
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.78	0.93	0.85	15
virginica	0.92	0.73	0.81	15
accuracy			0.89	45
macro avg	0.90	0.89	0.89	45
weighted avg	0.90	0.89	0.89	45

混淆矩阵第 i 行第 j 列表示“真实为 i 、被判为 j ”。`classification_report` 给出每类的 precision、recall、f1。对不平衡数据，准确率会误导，要看 f1/recall。

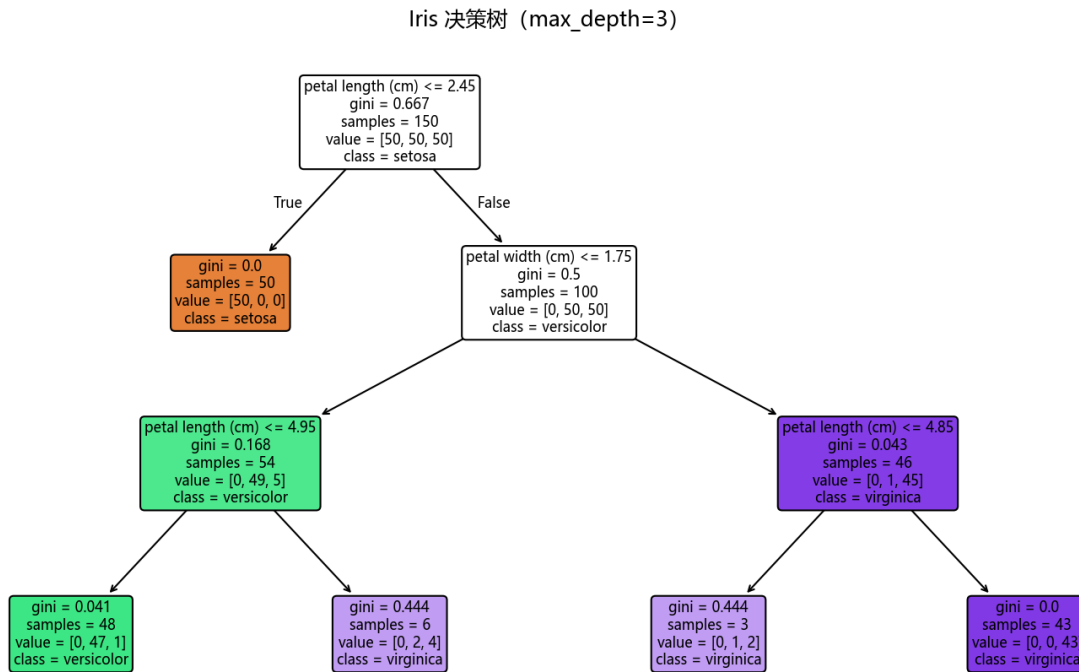


Figure 2: Iris 决策树

1.4 决策树可视化

```

import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
from sklearn.tree import plot_tree

dt = DecisionTreeClassifier(max_depth=3, random_state=42).fit(Xtr, ytr)
fig, ax = plt.subplots(figsize=(13, 7))
plot_tree(dt, feature_names=iris.feature_names,
          class_names=list(iris.target_names), filled=True, rounded=True, ax=ax,
          fontsize=8)
plt.title("Iris 决策树 (max_depth=3) ")

```

2. 二分类与 ROC-AUC: 乳腺癌

乳腺癌数据集是二分类 (良性/恶性)。逻辑回归对量纲敏感, 因此用 Pipeline 先标准化再训练。

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score

bc = load_breast_cancer(); Xb, yb = bc.data, bc.target
Xtr, Xte, ytr, yte = train_test_split(Xb, yb, test_size=0.3, random_state=42, stratify=yb)

pipe = Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression(max_iter=1000))])

```

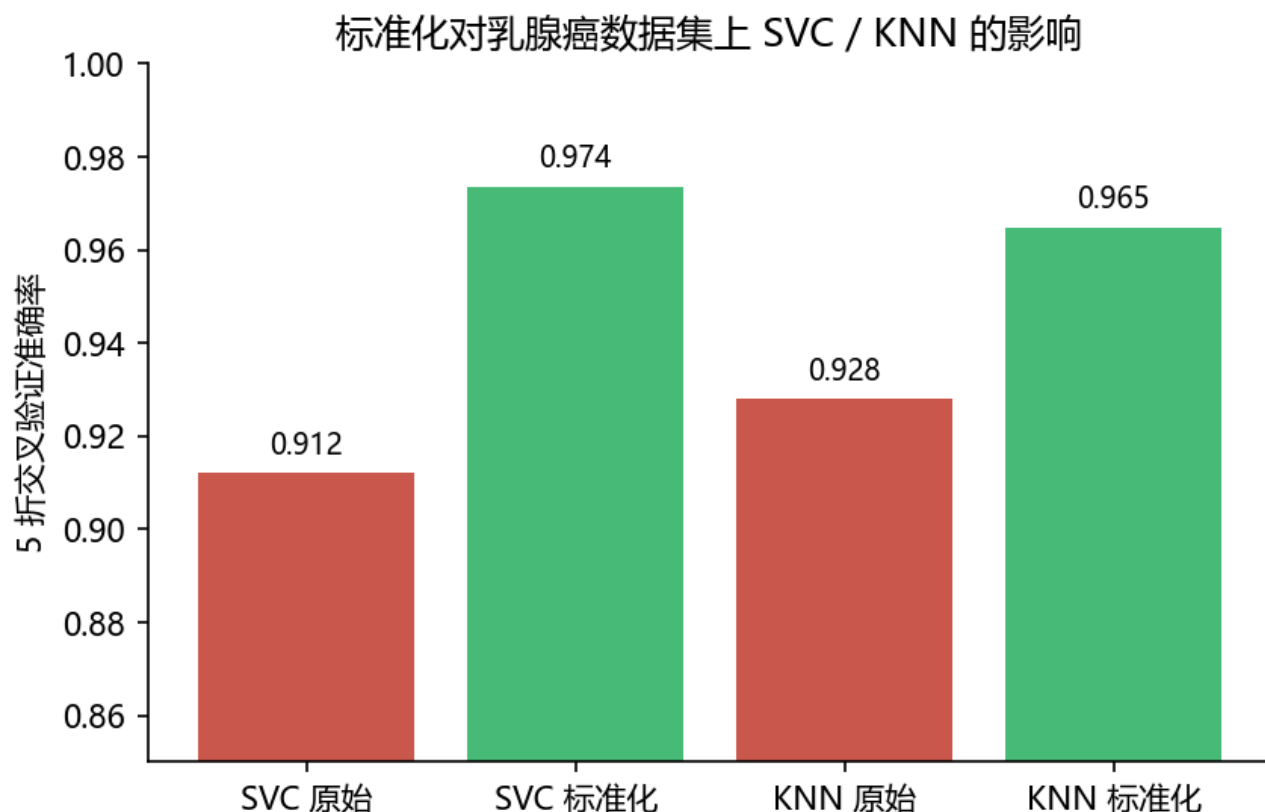


Figure 3: 缩放影响

```
pipe.fit(Xtr, ytr)
prob = pipe.predict_proba(Xte)[: , 1]
print("ROC-AUC:", round(roc_auc_score(yte, prob), 4))
print(" 准确率:", round(accuracy_score(yte, pipe.predict(Xte)), 4))
```

ROC-AUC: 0.9981

准确率: 0.9883

`predict_proba` 返回每个样本属于正类的概率；`roc_auc_score` 评估“排序能力”，比单纯准确率更能反映二分类模型质量。

缩放为什么关键：SVC/KNN 对量纲很敏感。下图给出乳腺癌 5 折交叉验证，未标准化 vs 标准化的对比：

3. 回归：Diabetes 糖尿病

波士顿房价数据集 (`load_boston`) 已在 `sklearn 1.2` 被移除，故本节改用内置的 `load_diabetes` (442 个样本、10 个标准化后的特征)。

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np
```

```
d = load_diabetes(); Xd, yd = d.data, d.target
Xtr, Xte, ytr, yte = train_test_split(Xd, yd, test_size=0.2, random_state=42)
```

```

for name, m in [("LinearRegression", LinearRegression()),
                ("Ridge(alpha=1)", Ridge(alpha=1.0)),
                ("Lasso(alpha=0.1)", Lasso(alpha=0.1, max_iter=5000))]:
    m.fit(Xtr, ytr)
    yp = m.predict(Xte)
    print(f"{name:20s} MSE={mean_squared_error(yte, yp):.2f} "
          f"MAE={mean_absolute_error(yte, yp):.2f} R2={r2_score(yte, yp):.4f}")

```

```

las = Lasso(alpha=0.1, max_iter=5000).fit(Xtr, ytr)
print("Lasso 非零系数个数:", np.count_nonzero(las.coef_))
print(" 特征名:", d.feature_names)

```

```

LinearRegression      MSE=2900.19  MAE=42.79  R2=0.4526
Ridge(alpha=1)        MSE=3077.42  MAE=46.14  R2=0.4192
Lasso(alpha=0.1)      MSE=2798.19  MAE=42.85  R2=0.4719
Lasso 非零系数个数: 7

```

```
特征名: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
```

回归指标速记: MSE/MAE 越小越好; R2 越接近 1 越好。这里 Lasso 用 L1 正则把部分系数压到 0, 既得到不错 R^2 又做了特征选择。

案例卡 2: 分类流水线——Iris 用 KNN+ 标准化 +Pipeline 防数据泄漏

目标

把 StandardScaler 与 KNeighborsClassifier 串成一条 Pipeline, 在 Iris 上完成训练和 5 折交叉验证; 并对比“未缩放直接喂 KNN”, 体会缩放对距离类模型的影响, 以及为什么 Pipeline 能防止数据泄漏。

代码

```

import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

np.set_printoptions(precision=4, suppress=True)

iris = load_iris(); X, y = iris.data, iris.target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

pipe = Pipeline([('sc', StandardScaler()), ('knn', KNeighborsClassifier(n_neighbors=5))])
pipe.fit(Xtr, ytr)
print('Pipeline KNN acc:', round(accuracy_score(yte, pipe.predict(Xte)), 4))
print('5 折 CV:', np.round(cross_val_score(pipe, X, y, cv=5), 4))
print('5 折均值:', round(cross_val_score(pipe, X, y, cv=5).mean(), 4))

knn_raw = KNeighborsClassifier(n_neighbors=5).fit(Xtr, ytr)
print('未缩放 KNN acc:', round(accuracy_score(yte, knn_raw.predict(Xte)), 4))

print('pipeline steps:', pipe.named_steps)

```

```
print('scaler mean_', np.round(pipe.named_steps['sc'].mean_, 4))
```

运行结果（已运行核验）

```
Pipeline KNN acc: 0.9111
5 折 CV: [0.9667 0.9667 0.9333 0.9333 1.    ]
5 折均值: 0.96
未缩放 KNN acc: 0.9778
pipeline steps: {'sc': StandardScaler(), 'knn': KNeighborsClassifier()}
scaler mean_: [5.8733 3.0552 3.7848 1.2057]
```

讲解

一句话解释： Pipeline 就像一条自动流水线：先把特征“缩放”，再交给 KNN 分类；每一次交叉验证都会在训练折重新缩放，绝不偷看测试折。

- 1. `train_test_split(random_state=42, stratify=y)` 把 Iris 按类别等比例分成训练 (105) 与测试 (45)，固定随机种子保证可复现。
- 2. `Pipeline([('sc', StandardScaler()), ('knn', KNeighborsClassifier(n_neighbors=5))])` 定义两步：sc 缩放，knn 分类。调用 `pipe.fit(Xtr, ytr)` 时，先 fit 缩放器再 fit KNN，一步完成。
- 3. `pipe.predict(Xte)` 会自动用训练集的均值和标准差缩放测试集，再交给 KNN 分类；测试集信息完全没有参与缩放统计量。
- 4. `cross_val_score(pipe, X, y, cv=5)` 每次折叠都会**重新 fit 整条流水线**，因此不会用整份数据算缩放器；5 折均值为 0.96，比单次测试集更可信。
- 5. 最后一行 `scaler mean_` 显示的是训练集均值 `[5.8733, 3.0552, 3.7848, 1.2057]`，它与 Case 1 中“只 fit 训练集”的结果一致，说明 Pipeline 没有泄漏。
- 6. 对比“未缩放 KNN acc=0.9778”：**在这次固定切分上未缩放反而略高**；但距离类模型对量纲敏感，单次 test 很随机，因此应看 5 折均值 (0.96) 或换多组切分，而不是凭一次结果下结论。

主要用法 / API

需求	写法	说明
拼接缩放 + 模型	<code>Pipeline([('sc', StandardScaler()), ('knn', KNeighborsClassifier())])</code>	fit 时自动先缩放再训练
交叉验证	<code>cross_val_score(pipe, X, y, cv=5)</code>	每折重新 fit 整条流水线
查看每一步	<code>pipe.named_steps</code>	返回名字到对象的字典
取某一步参数	<code>pipe.named_steps['sc'].mean_</code>	例如看缩放器统计量
预测概率	<code>pipe.predict_proba(X)[:, 1]</code>	二分类正类概率在第二列

常见错误

- 先 `StandardScaler().fit_transform(X)` 再切分 → 数据泄漏；应把缩放放进 Pipeline 或只对训练集 fit。
- 在 `cross_val_score` 里直接传缩放后的 X，而不是传 Pipeline → 等价于每折都用全量统计量。
- 认为“缩放一定提升精度”：单次 test 可能相反，但距离类模型通常更稳定；要用 CV 判断。

- 忘记设置 `random_state`: KMeans/KNN 等虽确定, 但数据切分与后续调参不设种子会无法复现。

拓展 / 跟练

1. 把 KNN 换成 `SVC(kernel='rbf')` 或 `LogisticRegression`, 重跑 Pipeline 5 折, 比较均值。
2. 用 `GridSearchCV` 搜索 `knn__n_neighbors`, 注意参数名前缀 `knn__`。
3. 把 `StandardScaler` 换成 `MinMaxScaler`, 观察 5 折均值是否变化。
4. 对测试集单独 `fit_transform`, 观察 ROC-AUC 或准确率是否被高估, 并写下结论。

常见误区

1. 用 `r2_score(y_pred, y_test)` 参数顺序写反 → 应为 `r2_score(y_true, y_pred)`; 对分类问题更不该用 R^2 。
2. 在 `sklearn` 里已经移除 `load_boston`, 再用会 `ImportError`。
3. 直接拿未缩放数据喂 `SVC/KNN/LogisticRegression`, 会导致收敛慢或精度差。
4. 把 `predict_proba` 的第一列当作正类概率 → 正类是第 1 列 (标签为 1 的类)。
5. 在同一个数据集上反复 “调参、看测试集”, 相当于把测试集当训练信息用。

思考题

1. 为什么 KNN 对缩放特别敏感? 树模型为什么相对不敏感?
2. `accuracy_score` 与 `roc_auc_score` 分别衡量什么? 在类别不平衡时哪个更可靠?
3. 一次 test 集准确率高, 能说明模型泛化好吗? 为什么还要交叉验证?

动手练习

1. 在 Iris 上把 `RandomForestClassifier(n_estimators=100)` 与 `KNeighborsClassifier` 做 5 折交叉验证比较。
2. 用 `SVC` 分别在不缩放/缩放情况下做乳腺癌 5 折, 记录结果 (可对照 `scaling_effect.png`)。
3. 在 Diabetes 上试 `Ridge(alpha)` 与 `Lasso(alpha)` 的不同 `alpha` 值, 画出 “alpha- R^2 ” 曲线。

延伸阅读

- `sklearn` 分类指标: [链接](#)
- `sklearn` 回归指标: [链接](#)
- `sklearn` 用户指南 “选择正确的评估器”: [链接](#)

03 无监督学习的案例

本节讲“没有标签”的任务：**聚类**（KMeans、DBSCAN）与**降维**（PCA、LDA）。所有代码在 sklearn 1.6.1 下验证。

本节目标

1. 用 KMeans 对 make_blobs 聚类，读出 inertia_、cluster_centers_、silhouette_score。
2. 用 DBSCAN 对 make_moons 聚类，识别噪声（标签 -1），并算 adjusted_rand_score。
3. 用 PCA 降维并读出 explained_variance_ratio_。
4. 用 LDA 做有监督降维，理解它与 PCA 的差别。

先修

- 01 节特征缩放、02 节分类/回归基础。
- 了解“距离”“方差”“协方差”等概念。

官方文档 / 参考入口

- sklearn 聚类：[链接](#)
- sklearn 降维：[链接](#)
- sklearn 聚类评估：[链接](#)

正文

1. 聚类：KMeans

KMeans 把数据分成 K 个簇，使“簇内平方和”最小。它需要预先指定 K。

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)
km = KMeans(n_clusters=4, random_state=0).fit(X)
labels = km.labels_
print("inertia:", round(km.inertia_, 4))
print(" 聚类中心形状:", km.cluster_centers_.shape)
print(" 轮廓系数:", round(silhouette_score(X, labels), 4))

inertia: 212.006
聚类中心形状: (4, 2)
轮廓系数: 0.682
```

inertia_ 是簇内平方和，越小越好；silhouette_score 在 [-1,1] 之间，越接近 1 表示簇内紧凑、簇间分离越好。

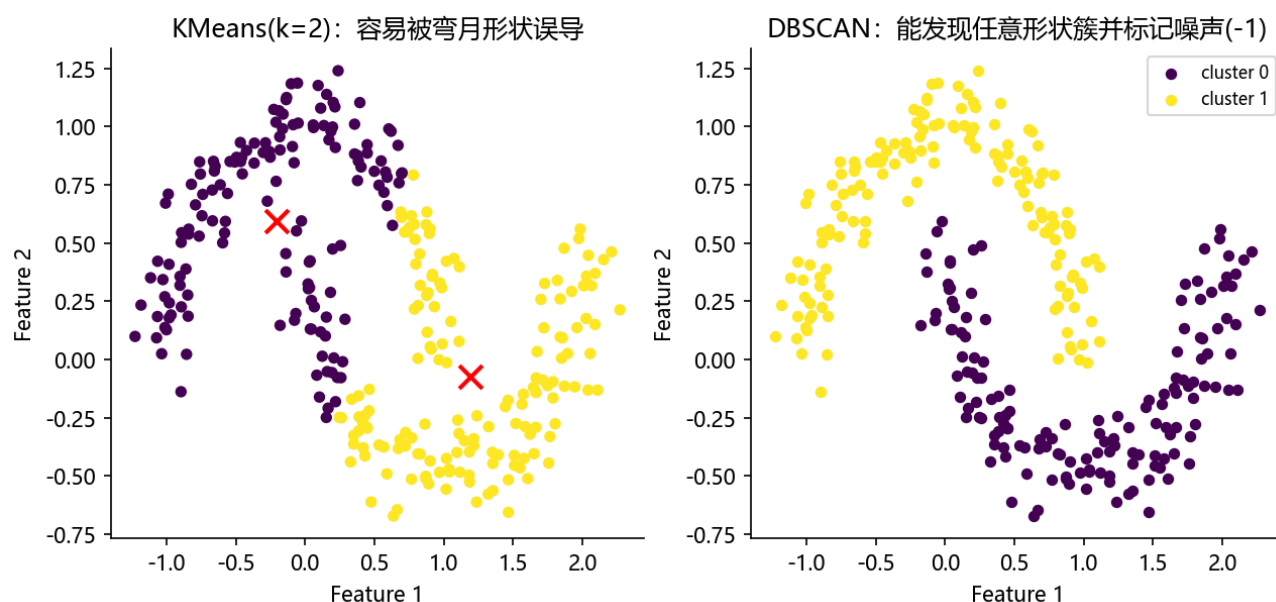


Figure 1: KMeans vs DBSCAN

2. 聚类: DBSCAN

DBSCAN 按密度自动发现簇，能处理任意形状，并能把不属于任何簇的点标为 -1 (噪声)。

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
from sklearn.metrics import adjusted_rand_score

Xm, ym = make_moons(n_samples=300, noise=0.1, random_state=42)
db = DBSCAN(eps=0.2, min_samples=5).fit(Xm)
labels = db.labels_
print(" 簇数:", len(set(labels)) - (1 if -1 in labels else 0))
print(" 噪声比例:", round(np.mean(labels == -1), 4))
print(" 簇大小:", np.bincount(labels))
print(" 与真实标签 ARI:", round(adjusted_rand_score(ym, labels), 4))
```

```
簇数: 2
噪声比例: 0.0
簇大小: [150 150]
与真实标签 ARI: 1.0
```

adjusted_rand_score (ARI) 在 [0,1] (随机约 0、完全一致为 1)。这里 DBSCAN 完美分出两个月牙形簇，ARI=1.0。

KMeans 与 DBSCAN 的对比: KMeans 假设凸形、球形簇，对“弯月”类数据效果差；DBSCAN 依据密度，可发现任意形状并识别噪声。

3. 降维: PCA (无监督)

PCA 找“方差最大的方向”，把高维映射到低维，且不需要标签。

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
```

```
iris = load_iris(); X, y = iris.data, iris.target
pca = PCA(n_components=2)
Xp = pca.fit_transform(X)
print(" 各主成分解释方差比:", np.round(pca.explained_variance_ratio_, 4))
print(" 累计解释方差:", np.round(np.cumsum(pca.explained_variance_ratio_), 4))
```

各主成分解释方差比: [0.9246 0.0531]

累计解释方差: [0.9246 0.9777]

前 2 个主成分已保留约 97.77% 方差，说明 Iris 降到 2 维几乎不丢信息，适合可视化。

4. 降维：LDA（有监督）

LDA 在已知类别的情况下，找“类间方差最大、类内方差最小”的投影方向。因此需要同时给出 X 与 y。

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
```

```
lda = LDA(n_components=2)
Xl = lda.fit_transform(X, y)
print("LDA 各成分解释方差比:", np.round(lda.explained_variance_ratio_, 4))
```

LDA 各成分解释方差比: [0.9912 0.0088]

对比：PCA 完全不看类别，LDA 用类别信息，因而投影对分类更有帮助。两者都适用于 Iris 降到 2 维，但语义不同。

案例卡 3：KMeans 与 PCA——用肘部法则评估聚类数并做 2D 可视化

目标

在 Wine 这个 13 维数值数据集上，先标准化，再用**肘部法则**（簇内平方和 inertia 随聚类数 k 变化）评估该选几个簇；最后用 **PCA** 把 13 维压到 2 维，画出 KMeans 聚类结果的 2D 可视化。

代码

```
import numpy as np
import os
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, adjusted_rand_score

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

wine = load_wine(); X = wine.data; y = wine.target
Xs = StandardScaler().fit_transform(X)
ks = range(1, 9)
inertias = []
for k in ks:
    km = KMeans(n_clusters=k, random_state=42).fit(Xs)
    inertias.append(km.inertia_)
```

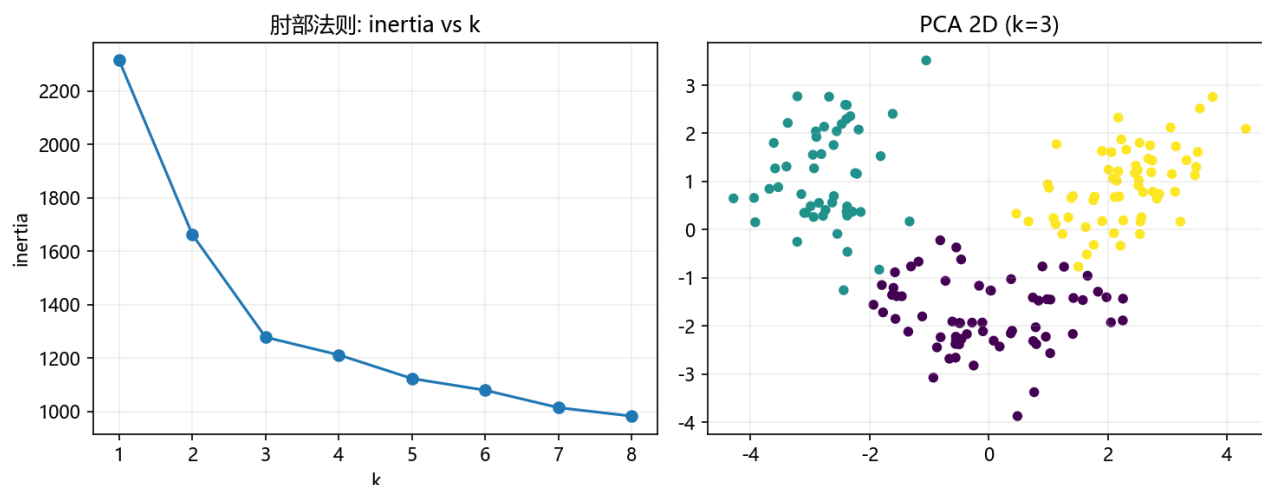


Figure 2: KMeans 肘部法则与 PCA 2D 可视化

```
print('inertia:', np.round(np.array(inertias), 2))
for k in [3, 4, 5]:
    km = KMeans(n_clusters=k, random_state=42).fit(Xs)
    print('k=%d silhouette=%.4f ARI=%.4f' % (k, silhouette_score(Xs, km.labels_),
        adjusted_rand_score(y, km.labels_)))

km3 = KMeans(n_clusters=3, random_state=42).fit(Xs)
pca = PCA(n_components=2)
Xp = pca.fit_transform(Xs)
print('PC1/PC2 var ratio:', np.round(pca.explained_variance_ratio_, 4))

os.makedirs('images', exist_ok=True)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(list(ks), inertias, 'o-')
axes[0].set_title('肘部法则: inertia vs k')
axes[0].set_xlabel('k'); axes[0].set_ylabel('inertia')
axes[1].scatter(Xp[:, 0], Xp[:, 1], c=km3.labels_, cmap='viridis', s=20)
axes[1].set_title('PCA 2D (k=3)')
for ax in axes:
    ax.grid(alpha=0.2)
fig.tight_layout()
fig.savefig('images/case3_elbow_pca.png', dpi=150)
plt.close()
print('saved images/case3_elbow_pca.png')
```

运行结果 (已运行核验)

```
inertia: [2314.  1661.68 1277.93 1211.75 1123.16 1079.54 1014.43  982.65]
k=3 silhouette=0.2849 ARI=0.8975
k=4 silhouette=0.2542 ARI=0.8172
k=5 silhouette=0.1836 ARI=0.6264
PC1/PC2 var ratio: [0.362  0.1921]
saved images/case3_elbow_pca.png
```

讲解

一句话解释：KMeans 要告诉它“分成几堆”，肘部法则看“再分一堆能省多少距离”，省得开始变少时的 k 就是不错的分堆数；PCA 则把 13 维投影到平面上，让我们能“看见”这些堆。

- 1. `StandardScaler().fit_transform(X)` 先对 13 个特征做 z-score，避免量纲大的特征（如 `proline`）主导聚类。
- 2. `KMeans(n_clusters=k, random_state=42)` 用固定随机种子，保证每次结果可复现；`inertia_` 是簇内平方和（WCSS），k 越大通常越小。
- 3. 肘部法则：观察 `inertia` 序列，k 从 1 到 2、2 到 3 下降很大（2314→1661→1278），3 之后下降明显变缓（1211、1123、1079...），所以“肘”在 k=3 附近。
- 4. 辅助指标：`silhouette`（越接近 1 越好）在 k=3 时最大（0.2849）；ARI 衡量与真实品种标签的一致性，k=3 时 ARI=0.8975，说明分成 3 簇与真实品种高度吻合。
- 5. `PCA(n_components=2)` 给出前两个主成分解释方差比例 [0.362, 0.1921]，两者合计约 0.554，虽不足 100%，但 2D 散点已能看出三团结构。
- 6. 右图按 KMeans 的簇标签着色，三团颜色基本对应真实的三类 Wine；左图（肘部图）则直观显示“k=3 是下降变缓的拐点”。

主要用法 / API

需求	写法	说明
标准化	<code>StandardScaler().fit_transform(X)</code>	距离/聚类前常用
聚类	<code>KMeans(n_clusters=k, random_state=42)</code>	需要固定随机种子
簇内平方和	<code>km.inertia_</code>	画肘部曲线
轮廓系数	<code>silhouette_score(Xs, km.labels_)</code>	越大越“内紧外分”
与真实标签比对	<code>adjusted_rand_score(y, km.labels_)</code>	1 为完全一致
降维到 2 维	<code>PCA(n_components=2).fit_transform(Xs)</code>	用于可视化

常见错误

- 不缩放就聚类：量纲大的特征会把其他特征“压扁”，常见结果是簇形状被大数值特征主导。
- 只看 `inertia` 就选 k： `inertia_` 只减不增，若没有明显“肘”，还要结合 `silhouette` 或业务知识。
- 用一次 KMeans 标签就下结论：KMeans 只是无监督分组，标签顺序与真实类别不一定对应，应结合 ARI/可视化。
- 把 `explained_variance_ratio_` 的累计值当成“模型准确率”：它是投影保留的方差比例，不是分类/聚类精度。

拓展 / 跟练

- 1. 把 ks 范围改为 1~10，看 k 更大时 `inertia` 是否还在“加速下降”。
- 2. 把标准化后的 Wine 改用 DBSCAN，比较它对 Wine 的簇数与 ARI。
- 3. 用 PCA 的 `explained_variance_ratio_` 画“累计解释方差”图，判断保留到 2 维是否够。
- 4. 对 Iris 也做同样的肘部图 + PCA 2D 可视化，比较 Wine 与 Iris 的最佳 k 是否都为 3。

常见误区

- 1. KMeans 必须预设 `n_clusters`，且对初始中心敏感（用 `random_state` 固定，或用 `k-means++`）。
- 2. 以为“降维能提升所有模型精度”——降维会丢失信息，有时反而更差。

3. 把 PCA 与 LDA 混为一谈（前者无监督、后者有监督）。
4. 对 `make_moons` 用 KMeans 期望得到月牙形簇——通常失败。
5. DBSCAN 的 `eps`、`min_samples` 必须按数据量纲调，不能 “一个参数跑天下”。

思考题

1. 为什么 KMeans 需要先缩放？PCA 呢？
2. DBSCAN 的 `eps` 太小/太大会分别出现什么现象？
3. 用什么指标判断 “该保留多少个主成分”？（累计解释方差、碎石图等）
4. ARI 与 `silhouette_score` 的适用场景有何不同？

动手练习

1. 对 `make_blobs` 用 `KMeans(k=2..6)` 画出 “k-轮廓系数” 曲线，找最佳 k。
2. 调整 DBSCAN 的 `eps` (0.1/0.2/0.5)，观察簇数与噪声比例变化。
3. 对 Wine 数据（13 维）用 PCA 降到 2 维并散点着色，给出累计解释方差。

延伸阅读

- sklearn 聚类用户指南：[链接](#)
- sklearn 降维用户指南：[链接](#)
- sklearn 聚类评估：[链接](#)

04 综合案例：Wine 数据端到端（预处理 → 分类 → 聚类 → 降维）

本章新增综合案例，把 01-03 的知识串成一条完整流水线：**特征缩放** → **分类** → **特征重要性** → **聚类** → **降维** → **可视化**。本综合案例按“案例卡 4”组织：目标 → 步骤 → 讲解 → 主要用法 / API → 常见错误 → 拓展练习。所有代码在 sklearn 1.6.1 下验证；配图由 build/make_chap8_figures.py 生成。

案例背景

Wine（红酒）数据集有 178 个样本、13 个数值特征（酒精、苹果酸、灰分、总酚、黄酮等），标签为 3 个品种（class_0/1/2）。这是一个“数值特征多、量纲不同、类别有结构”的典型数据，非常适合练习本章全部知识。

案例目标

- 用 StandardScaler + Pipeline 完成“缩放 + 分类”，在训练/测试切分与 5 折交叉验证中都避免数据泄漏；
- 用随机森林 feature_importances_ 找出区分品种的关键特征；
- 用 KMeans 做无监督聚类，并用 silhouette_score / adjusted_rand_score 评估聚类质量；
- 用 PCA 降到 2 维并可视化，比较“真实类别”与“KMeans 簇”在平面上的重合程度。

第 1 步：加载数据并观察

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

wine = load_wine()
X, y = wine.data, wine.target
print(" 特征名:", wine.feature_names)
print(" 数据形状:", X.shape, " 类别:", np.unique(y))

特征名: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols',
'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity', 'hue',
'od280/od315_of_diluted_wines', 'proline']
数据形状: (178, 13) 类别: [0 1 2]
```

第 2 步：缩放 + 分类（用 Pipeline 防泄漏）

把“标准化”与“分类器”放进 Pipeline，这样 cross_val_score 每次都在训练折上拟合缩放器，绝不用测试折的信息。

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
print("train:", Xtr.shape, "test:", Xte.shape)

lr_pipe = Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression(max_iter=1000))])
rf_pipe = Pipeline([("sc", StandardScaler()), ("clf",
RandomForestClassifier(n_estimators=100, random_state=42))])

lr_pipe.fit(Xtr, ytr)
rf_pipe.fit(Xtr, ytr)
print("LogisticRegression test acc:", round(accuracy_score(yte, lr_pipe.predict(Xte)), 4))
print("RandomForest test acc:", round(accuracy_score(yte, rf_pipe.predict(Xte)), 4))
print("LogisticRegression 5 折均值:", round(cross_val_score(lr_pipe, X, y, cv=5).mean(),
4))
print("RandomForest 5 折均值:", round(cross_val_score(rf_pipe, X, y, cv=5).mean(), 4))

train: (124, 13) test: (54, 13)
LogisticRegression test acc: 0.9815
RandomForest test acc: 1.0
LogisticRegression 5 折均值: 0.9832
RandomForest 5 折均值: 0.9778

```

真实项目里，一组固定的 test 集被反复调参会导致“测试集过拟合”；所以更稳的是看 5 折平均（这里 LogisticRegression 与 RandomForest 均约 0.98）。

第 3 步：特征重要性 (RandomForest)

用随机森林的 `feature_importances_` 看哪些特征更重要。

```

from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

Xs = StandardScaler().fit_transform(X)
rf = RandomForestClassifier(n_estimators=100, random_state=42).fit(Xs, y)
imp = rf.feature_importances_
order = np.argsort(imp)[::-1]
for i in order[:6]:
    print(f"{wine.feature_names[i]:30s} {imp[i]:.4f}")

flavanoids           0.1945
color_intensity      0.1730
alcohol              0.1416
proline              0.1370
od280/od315_of_diluted_wines 0.1118
hue                  0.0769

```

黄酮 (flavanoids)、颜色强度 (color_intensity)、酒精含量 (alcohol)、脯氨酸 (proline) 等对品种区分贡献最大。这有助于我们“读懂”模型而不仅是得到一个分数。

第 4 步：无监督聚类 (KMeans)

把“没有标签”的思路也用上：对标准化后的 Wine 做 KMeans，看能否大致还原真实品种。

```

from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score, silhouette_score

```

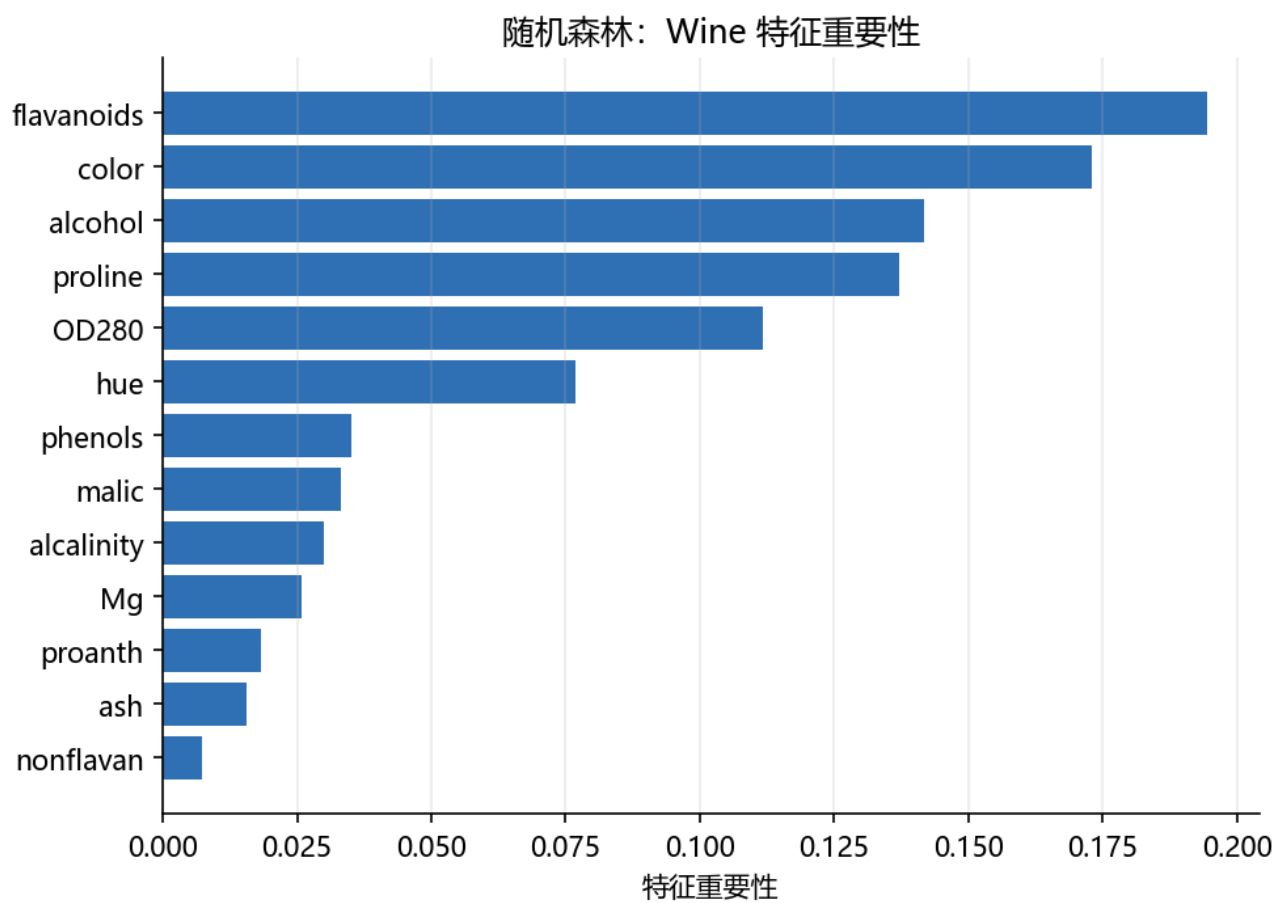


Figure 1: Wine 特征重要性

```
km = KMeans(n_clusters=3, random_state=42).fit(Xs)
print(" 轮廓系数:", round(silhouette_score(Xs, km.labels_), 4))
print("ARI vs 真实标签:", round(adjusted_rand_score(y, km.labels_), 4))
print(" 各簇大小:", np.bincount(km.labels_))
```

轮廓系数: 0.2849

ARI vs 真实标签: 0.8975

各簇大小: [65 51 62]

adjusted_rand_score=0.8975 表示无监督聚类与真实品种高度一致（1 为完全一致），说明 Wine 数据确有明显的类结构。

第 5 步：PCA 降维并可视化

用 PCA 把 13 维压到 2 维，分别按真实类别与 KMeans 簇着色：

```
import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
Xp = pca.fit_transform(Xs)
print(" 前两个主成分解释方差:", np.round(pca.explained_variance_ratio_, 4))
print(" 累计解释方差:", np.round(np.cumsum(pca.explained_variance_ratio_), 4))
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(Xp[:, 0], Xp[:, 1], c=y, cmap="viridis", s=22)
axes[0].set_title("PCA 按真实类别着色")
axes[1].scatter(Xp[:, 0], Xp[:, 1], c=km.labels_, cmap="viridis", s=22)
axes[1].set_title("PCA 按 KMeans 簇着色")
```

```
for ax in axes:
    ax.set_xlabel("PC1"); ax.set_ylabel("PC2"); ax.grid(alpha=0.2)
```

```
fig.tight_layout()
```

```
# 图见 images/wine_pca_true_vs_kmeans.png
```

前两个主成分解释方差: [0.362 0.1921]

累计解释方差: [0.362 0.5541]

前两个主成分只保留约 55% 方差，说明 13 维信息没有完全集中在 2 维，但散点已能看出三团结构；KMeans 簇与真实类别在 PCA 平面上也基本重合。

案例讲解

一句话解释：把 13 个红酒指标当成 13 维坐标，先缩放拉平尺度，再训练分类器、看哪些指标最重要，最后用 KMeans 分堆并用 PCA 投到平面上一眼看清。

1. 第 1 步打印 wine.feature_names 与 X.shape，先“看清数据”。13 个特征量纲不同，所以后面必须做缩放。
2. 第 2 步用 Pipeline 把 StandardScaler 和分类器串起来；cross_val_score 时每折都会重新 fit 缩放器，避免信息泄漏。5 折均值：LogisticRegression=0.9832、RandomForest=0.9778。
3. 第 3 步查看 feature_importances_: flavanoids(0.1945)、color_intensity(0.1730)、alcohol(0.1416) 贡献最大，说明这几个指标最能区分品种。
4. 第 4 步 KMeans 聚类 ARI=0.8975，说明无监督分组与真实品种高度一致；轮廓系数 0.2849 说明簇间还有一定重叠。

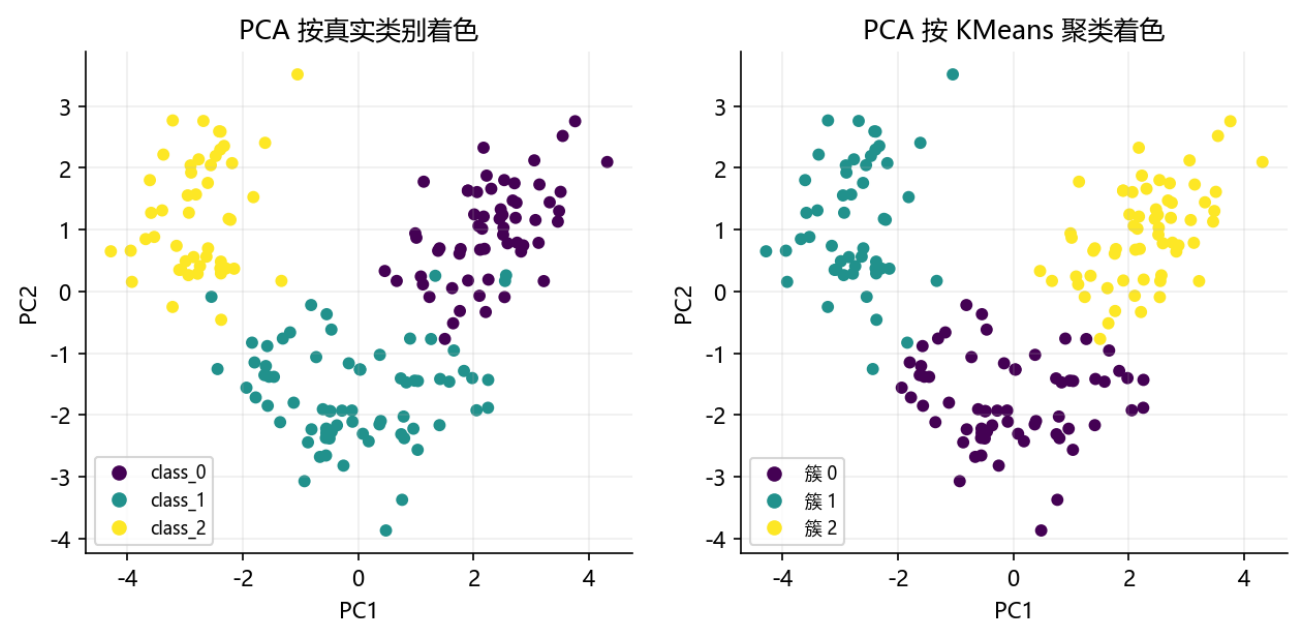


Figure 2: Wine PCA: 真实类别 vs KMeans 簇

5. 第 5 步 PCA 前两个主成分解释方差约 55%，2D 平面已能看出三团；按真实标签与按 KMeans 簇着色基本重合，说明这个数据在低维确实可分。

主要用法 / API

需求	写法	说明
缩放 + 分类	<code>Pipeline([('sc', StandardScaler()), ('clf', LogisticRegression())])</code>	训练/预测统一走流水线
交叉验证	<code>cross_val_score(pipe, X, y, cv=5)</code>	每折重新拟合
特征重要性	<code>rf.feature_importances_</code>	越大越重要
聚类评估	<code>silhouette_score</code> <code>adjusted_rand_score</code>	内紧外分 / 与真实标签一致性
降维	<code>PCA(n_components=2)</code>	读 <code>explained_variance_ratio_</code>

常见错误

- 在 `cross_val_score` 里把 “缩放后的 X” 传进去，而不是传 Pipeline → 数据泄漏。
- 直接用未缩放的 Wine 做 KMeans → 量纲大的特征主导，簇可能失衡。
- 把 PCA 的 “累计解释方差” 当成分类精度 → 它是投影保留的方差比例。
- 只凭一次 train/test 下结论：小数据要结合 5 折均值。

本章综合练习（上机任务）

1. 把本案例中的分类器换成 SVC、GradientBoostingClassifier，比较 5 折均值。
2. 用 SelectKBest 只保留前 5 个特征再分类，观察精度是否下降。

3. 把 KMeans 的 `n_clusters` 改为 2、4、5, 画出 “k-ARI” 曲线, 解释最佳 k。
4. 把 PCA 画成 2×2 子图 (PC1-PC2、PC1-PC3、PC2-PC3), 观察哪个组合最易分。

延伸阅读

- sklearn 组合与 Pipeline: [链接](#)
- sklearn 特征重要性: [链接](#)
- sklearn 聚类评估 (ARI/silhouette): [链接](#)

05 常见误区与技巧

本章新增“查漏补缺”页：把 sklearn 使用中最常见的坑集中列出，并给出性能、调试与自测清单。

1. 高频误区速查表

#	误区 / 错误写法	正确做法	原因
1	<code>fit_transform(X_test)</code>	训练集 <code>fit</code> ，然后训练/测试都 <code>transform</code> ；或用 <code>Pipeline</code>	用测试集统计量会“泄漏”测试信息
2	对无序分类特征用 <code>LabelEncoder</code>	用 <code>OneHotEncoder</code> / <code>OrdinalEncoder</code>	整数编码隐含顺序，误导模型
3	<code>r2_score(y_pred, y_test)</code>	<code>r2_score(y_test, y_pred)</code>	第一个参数是真值 <code>y_true</code>
4	分类问题用 <code>r2_score</code>	用 <code>accuracy_score</code> / <code>f1_score</code> / <code>roc_auc_score</code>	R^2 只用于连续回归
5	<code>fit_transform</code> 后再 <code>fit</code> 另一模型	把缩放放进 <code>Pipeline</code> 统一处理	避免重复/不一致的预处理
6	<code>load_boston()</code>	用 <code>load_diabetes</code> / <code>fetch_california_housing</code> / 自定义数据	<code>load_boston</code> 已在 sklearn 1.2 移除
7	<code>OneHotEncoder(sparse=False)</code>	<code>sparse_output = False</code>	新版本用 <code>sparse_output</code>
8	<code>RFE(estimator, 5)</code> 传位 置参数	<code>RFE(estimator, n_features_to_select=5)</code>	新版 <code>n_features_to_select</code> 是关键字参数
9	用 <code>fit</code> 后直接 <code>transform</code> 新数据	先 <code>fit</code> 再 <code>transform</code> ，否则 <code>NotFittedError</code>	估计器必须携带训练统计量
10	<code>GridSearchCV</code> 不设 <code>random_state</code>	给模型与 <code>shuffle</code> 设置种子	结果不可复现
11	对树模型也强制缩放	通常可跳过（缩放对树无益）	树按阈值切分，不受量纲影响
12	DBSCAN 一个 <code>eps</code> 走天下	按数据量纲调 <code>eps/min_samples</code>	密度参数与数据尺度强相关
13	KMeans 不设 <code>random_state</code>	<code>KMeans(..., random_state=0)</code>	初始中心随机，结果不稳定
14	用 <code>predict</code> 取概率	概率用 <code>predict_proba</code> （二分类取 <code>[:,1]</code> ）	<code>predict</code> 返回类别标签
15	混淆矩阵行/列方向记反	记住 <code>confusion_matrix(y_true, y_pred)</code> 行是真值	行 = 真实，列 = 预测

2. 性能技巧

- 数据量优先用 `n_jobs=-1`: `RandomForestClassifier(n_jobs=-1)`、`GridSearchCV(..., n_jobs=-1)` 可并行（注意 RAM）。
- 能向量化别循环：特征处理尽量用 sklearn 方法一次性完成，避免 Python 逐行 `for`。
- 大矩阵用稀疏表示：OneHot 后特征维度很大时保留稀疏矩阵（默认 `sparse_output=True`），别急着 `toarray()`。

- **Pipeline 更好维护**：把缩放、编码、特征选择、模型串成一个对象，训练/预测/调参都更安全。
- **网格搜索先粗后细**：先搜大跨度，再在小范围精调；必要时用 RandomizedSearchCV 代替 GridSearchCV。
- **启用 Warm start**：增量训练可用 partial_fit (如 SGDClassifier) 或 warm_start=True (部分集成模型)。

3. 调试建议

1. 先打印 X.shape、y.shape、dtype，确认维度与缺失值。
2. 用 np.isnan(X).sum()、np.isinf(X).sum() 检查异常值。
3. 出错先看 **fit 是否成功**：model.fit(X, y) 后打印 model.coef_/model.feature_importances_。
4. 对 Pipeline，用 pipe.named_steps 查看每一步；用 pipe.get_params() 检查参数名。
5. 检查是否“数据泄漏”：
 - 缩放器是否只 fit 在训练折（用 Pipeline 最保险）；
 - train_test_split 是否在归一化之前。
6. 收敛警告（如 “lbfgs failed to converge”）通常提示：特征未缩放、max_iter 太小，或类别不平衡。
7. 固定所有随机种子（random_state）并记录，便于复现与讨论。

4. 本章小结（自测）

- ☐ 能说清 fit / transform / fit_transform 的区别；
- ☐ 能用 Pipeline 做“缩放 + 模型”，避免数据泄漏；
- ☐ 能区分分类（准确率/F1/AUC）与回归（MSE/MAE/R²）指标；
- ☐ 能解释 StandardScaler 与 MinMaxScaler 的差异；
- ☐ 能比较 KMeans 与 DBSCAN 的适用场景；
- ☐ 能用 PCA / LDA 降到 2 维并读 explained_variance_ratio_；
- ☐ 能独立完成 04 综合案例并写 3 句结论。

延伸阅读

- sklearn API 速查：[链接](#)
- sklearn 常见陷阱（用户指南）：[链接](#)
- sklearn 调试/性能相关：[链接](#)